AsistenteIA — Documentación complementaria

Asistente académico móvil con inteligencia artificial · .NET MAUI, API REST y SQLite

Juan Carlos Díaz Cifuentes

Sebastián Grande Moreno

Universidad de San Buenaventura · Tecnología en Desarrollo de Software · VI semestre · Programación Avanzada y Aplicada · Docente: Sandra Esperanza Sánchez Barragán · Noviembre de 2026

Tabla de contenido

00. Comparación: documento entregado vs. lista de requisitos....3

01. Diagrama de arquitectura del sistema....7

02. Diagrama de clases....9

03. Modelo entidad-relación (SQLite)....12

04. Diccionario de datos....14

05. Documentación de la API REST....16

06. Arquitectura de proveedores de IA....19

07. Gestión de API Keys....21

08. Gestión de errores y excepciones....23

09. Requisitos técnicos actualizados....26

10. Auditoría de seguridad de API Keys....28

11. Manual de usuario (actualizado)....31

12. Manual técnico para desarrolladores....34

13. Manual de solución de problemas....36

14. Actualización de tecnologías utilizadas....38

15. Actualización de funcionalidades....40

16. Historial de cambios del proyecto....42

17. Evidencia del repositorio Git..... 44

18. Conclusiones y trabajo futuro.....46

Este documento completa la «Documentación del proyecto final» (octubre de 2026) con los 18 entregables solicitados y registra las diferencias encontradas entre ese documento y el código real del proyecto. Las versiones en Markdown están en el repositorio, en docs/complementaria/.

00. Comparación: documento entregado vs. lista de requisitos

Se comparó el documento **«AsistenteIA – Documentación del proyecto final» (PDF, 21 páginas, octubre de 2026)** con los 18 entregables solicitados y con el **código real** del proyecto (AsistenteIA_mejorado.zip).

Convención: ✅ completo · ⚠️ parcial o desactualizado · ❌ no existe en el documento.

#	Entregable solicitado	Estado en el PDF	Qué tenía el PDF	Qué faltaba o estaba mal	Dónde quedó completo
1	Diagrama de arquitectura del sistema	❌	Tabla de carpetas (§3.1) y recorrido de una consulta en texto (§3.2).	No hay representación visual de componentes ni de la comunicación app ↔ API de IA ↔ SQLite.	**01-Diagrama-Arquitectura**
2	Diagrama de clases	❌	Lista de clases por carpeta.	No hay atributos, métodos, responsabilidades ni relaciones.	**02-Diagrama-Clases**
3	Modelo entidad-relación (ERD)	⚠️	Una entidad (§6.1).	Falta la tabla **Project** y su relación; el campo se llama «Proyecto» en vez de ProjectId; falta Note; no hay PK/FK ni cardinalidad.	**03-Modelo-Entidad-Relacion**
4	Diccionario de datos	⚠️	Campo / tipo / descripción de una tabla.	Sin restricciones (longitudes, nulos, valores por defecto), sin tabla Project, sin claves de Preferences y SecureStorage.	**04-Diccionario-Datos**
5	Documentación de la API REST	⚠️	Solicitud, respuesta y códigos de Gemini (§5).	Faltan parámetros (generationConfig.maxOutputTokens), cuerpo de error, finishReason/promptFeedback, códigos 402 y 408, y la API de Pollinations.	**05-API-REST**
6	Arquitectura de proveedores de IA (Gemini y Pollinations, cambio manual)	❌	Pollinations solo aparece como alternativa descartada (§3.4).	El código **no** tenía Pollinations ni selección de proveedor. **Se implementó** en el código y se documentó.	**06-Arquitectura-Proveedores-IA**

#	Entregable solicitado	Estado en el PDF	Qué tenía el PDF	Qué faltaba o estaba mal	Dónde quedó completo
7	Gestión de API Keys	⚠️	Menciones a SecureStorage, campo oculto y encabezado (§1, §3.4, §5.1).	No explica ciclo de vida (guardar, probar, eliminar), cifrado por plataforma ni una clave por proveedor.	**07-Gestion-API-Keys**
8	Gestión de errores y excepciones	⚠️	Tabla de códigos (§5.3).	Faltaban **408** (tampoco estaba en el código) y **402**; no documenta las excepciones internas ni el flujo.	**08-Gestion-Errores**
9	Requisitos técnicos actualizados	⚠️	«.NET 9 o superior», carga de trabajo de VS, paquetes sin versión (§10.1).	Sin versiones exactas (SDK 9.0.100, MAUI, paquetes), sin versiones mínimas de Windows/Android, sin aviso de fin de soporte de .NET 9.	**09-Requisitos-Tecnicos**
10	Auditoría de seguridad de API Keys	❌	Una fila en la matriz de riesgos (§8.1).	No hay análisis de amenazas, verificación de controles ni hallazgos.	**10-Auditoria-Seguridad-API-Keys**
11	Manual de usuario actualizado	⚠️	Primeros pasos, consulta, historial, proyectos (§9).	No cubre: selección de proveedor, aviso de uso responsable, aviso de datos sensibles, límite de 8 consultas/min, notas, detalle de proyecto, borrar historial / todos los datos.	**11-Manual-Usuario**
12	Manual técnico para desarrolladores	⚠️	«Manual técnico breve» (§10).	Falta organización del código, cómo agregar un proveedor, ejecución de pruebas por consola, mantenimiento.	**12-Manual-Tecnico**
13	Manual de solución de problemas	❌	—	No existe.	**13-Solucion-Problemas**
14	Actualización de tecnologías utilizadas	⚠️	Tabla de tecnologías sin versiones (§1).	Sin versiones; no menciona xUnit, Pollinations ni la vigencia de los modelos Gemini.	**14-Actualizacion-Tecnologias**

#	Entregable solicitado	Estado en el PDF	Qué tenía el PDF	Qué faltaba o estaba mal	Dónde quedó completo
15	Actualización de funcionalidades	⚠️	RF01–RF10 «Cumple» (§2.2).	El código tiene funciones no descritas (8 pantallas, notas, límite de uso, datos sensibles, borrado de datos) y el PDF dice «diez casos» pero solo lista nueve (P01–P09).	**15-Actualizacion-Funcionalidades**
16	Historial de cambios del proyecto	❌	—	No existe.	**16-Historial-Cambios**
17	Evidencia del repositorio Git	⚠️	Párrafo «rama main con commits por funcionalidad de los dos integrantes» (§11).	**No coincide con el repositorio real**: el .git del zip tenía 3 commits de otro autor y un *merge* sin terminar; el repositorio de GitHub estaba vacío.	**17-Evidencia-Git**
18	Conclusiones y trabajo futuro	❌	—	No existe.	**18-Conclusiones-Trabajo-Futuro**

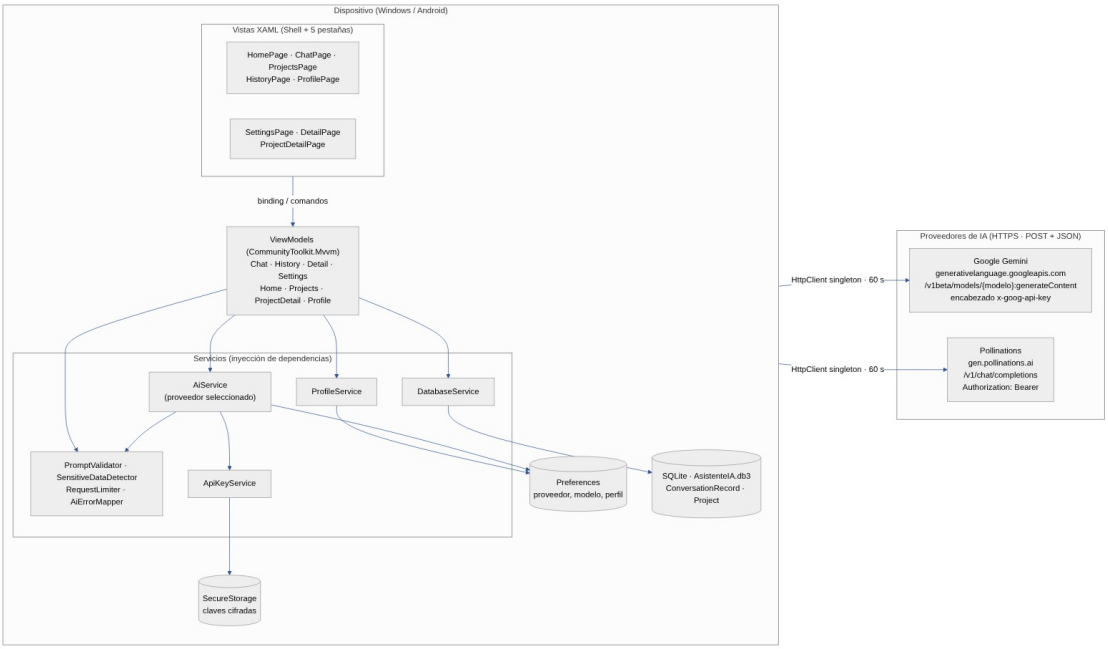
Resumen: 0 de 18 completos en el PDF · 11 parciales · 7 inexistentes. Todos quedan completos en esta carpeta.

Otras diferencias encontradas entre el PDF y el código

Tema	PDF	Código real	Acción
Pruebas automáticas	34 pruebas	34 en el zip	Se agregaron 32 → **66** (pendiente ejecutarlas en Visual Studio).
Casos de prueba manuales	«Diez casos»	Tabla con nueve (P01–P09); docs/03-Plan-de-pruebas.md define P01–P15	Documentado en 15-Actualizacion-Funcionalidades.
Autoría en «Acerca de»	Nombres de los integrantes	Decía «[escriba aquí los nombres del equipo]»	Corregido en SettingsPage.xaml.
Pantallas	7 (lista de cotejo)	8 (incluye Detalle de proyecto)	Documentado.

Tema	PDF	Código real	Acción
Error 408	No se menciona	No se manejaba	Agregado en AiErrorMapper.
Proveedor Pollinations	Alternativa descartada	No existía	Implementado con cambio manual en Configuración.
ApplicationId	—	com.companyname.asistenteia (valor de plantilla)	Recomendado cambiarlo antes de publicar (trabajo futuro).

01. Diagrama de arquitectura del sistema



Arquitectura del sistema

Fuente editable: diagramas/fuentes/arquitectura.mmd (Mermaid).

Componentes

Capa	Componentes	Responsabilidad	Se comunica con
Vistas (XAML + Shell)	5 pestañas: Inicio, Consultar, Proyectos, Historial, Perfil. Rutas secundarias: Configuración, Detalle, Detalle de proyecto.	Mostrar datos y capturar acciones. No hacen solicitudes HTTP.	ViewModels por *binding* y comandos.
ViewModels (MVVM)	HomeViewModel, ChatViewModel, ProjectsViewModel, ProjectDetailViewModel, HistoryViewModel, DetailViewModel, ProfileViewModel, SettingsViewModel.	Estado observable, validación, comandos.	Servicios inyectados por constructor.
Servicios	AiService, ApiKeyService, DatabaseService, ProfileService, INavigationService, IDialogService.	Acceso a la API de IA, almacenamiento seguro, SQLite, preferencias, navegación.	APIs externas y almacenes locales.
Lógica pura	PromptValidator, SensitiveDataDetector,	Reglas sin dependencia de MAUI; probadas con	—

Capa	Componentes	Responsabilidad	Se comunica con
	RequestLimiter, AiErrorMapper, AiProviderCatalog.	xUnit.	
Almacenamiento local	SQLite (AsistenteIA.db3), SecureStorage, Preferences.	Historial y proyectos; claves cifradas; proveedor, modelo y perfil.	Solo el dispositivo.
Proveedores de IA	Google Gemini · Pollinations.	Generar la respuesta.	HTTPS, POST + JSON, HttpClient único (60 s).

Comunicación

1. **Vista → ViewModel:** *data binding* ({Binding}) y comandos generados con [RelayCommand].
2. **ViewModel → AiService:** AskAsync(prompt, …, CancellationToken); asíncrono, no bloquea la interfaz.
3. **AiService → proveedor:** según el proveedor guardado en Preferences arma el POST de Gemini (generateContent) o de Pollinations (chat/completions). La clave se lee de SecureStorage y viaja en un encabezado.
4. **ViewModel → DatabaseService → SQLite:** al pulsar *Guardar*, *Verificar*, *Nota* o *Eliminar*.
5. **Inyección de dependencias** (MauiProgram.cs): servicios *singleton*, ViewModels y páginas *transient*.

02. Diagrama de clases

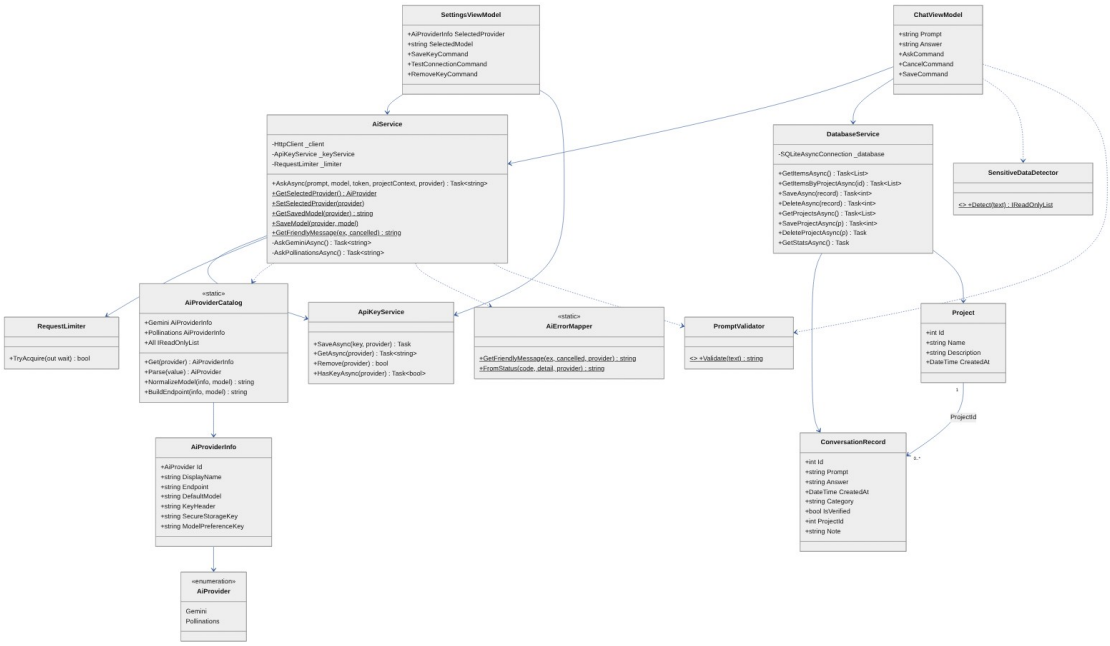


Diagrama de clases

Fuente editable: diagramas/fuentes/clases.mmd.

Clases principales

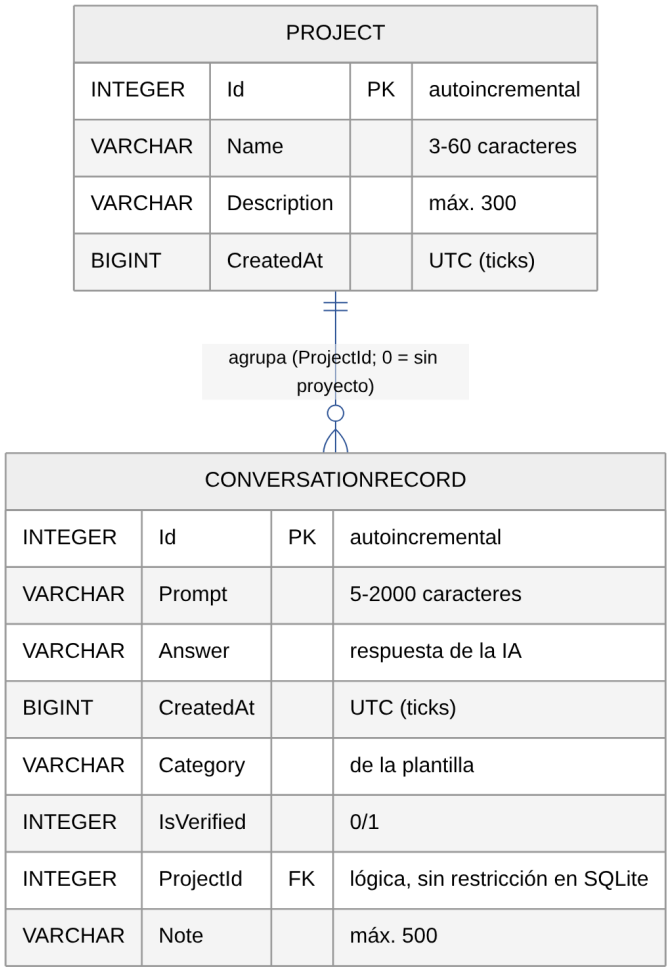
Clase	Tipo	Atributos / propiedades clave	Métodos clave	Responsabilidad
AiService	Servicio (singleton)	_client (HttpClient), _keyService, _limiter; constantes MaxOutputTokens = 4096, MaxPromptLength = 2000	AskAsync, GetSelectedProvider, SetSelectedProvider, GetSavedModel, SaveModel, GetFriendlyMessage	Validar, comprobar red/clave/límite y enviar la consulta al proveedor elegido.
AiProviderCatalog / AiProviderInfo / AiProvider	Lógica pura	Endpoint, modelo por defecto, modelos sugeridos, encabezado de clave, nombre en SecureStorage y Preferences	Get, Parse, NormalizeModel, BuildEndpoint	Describir cada proveedor en un solo lugar.
AiErrorMapper	Lógica pura (estática)	—	GetFriendlyMessage, FromStatus	Convertir excepciones y

Clase	Tipo	Atributos / propiedades clave	Métodos clave	Responsabilidad
				códigos HTTP en mensajes en español sin la clave.
ApiKeyService	Servicio	—	SaveAsync, GetAsync, Remove, HasKeyAsync (por proveedor)	Guardar la clave en SecureStorage.
DatabaseService	Servicio	_database (SQLiteAsyncConnection, inicialización perezosa)	CRUD de ConversationRecord y Project, GetStatsAsync	Persistencia local.
ProfileService	Servicio	Claves de Preferences	Load, Save, Clear, NoticeAccepted	Perfil del estudiante y aviso de uso responsable.
PromptValidator	Lógica pura	MinLength = 5, MaxLength = 2000	Validate	RF02.
SensitiveDataDetector	Lógica pura	Expresiones regulares con *timeout* de 200 ms	Detect, PassesLuhn	Avisar de correos, tarjetas, teléfonos, credenciales y claves.
RequestLimiter	Lógica	Máx. 8 solicitudes por minuto	TryAcquire(out wait)	Proteger la cuota de la clave.
ConversationRecord, Project	Entidades SQLite	Ver diccionario de datos	Propiedades de presentación con [Ignore]	Datos persistentes.
ChatModels	DTO JSON	ChatRequest/ ChatResponse (Gemini), OpenAiChatRequest/OpenAiChatResponse (Pollinations)	—	Serialización con System.Text.Json.
ViewModels	ObservableObject	[ObservableProperty]	[RelayCommand]	Estado y comandos de cada pantalla.

Relaciones

- **Composición por inyección:** ChatViewModel → AiService, DatabaseService, IDialogService; SettingsViewModel → ApiKeyService, AiService; AiService → HttpClient, ApiKeyService, RequestLimiter.
- **Dependencias de uso:** AiService usa AiProviderCatalog, PromptValidator y AiErrorMapper; ChatViewModel usa SensitiveDataDetector.
- **Asociación de datos:** un Project agrupa 0..* ConversationRecord mediante ProjectId (0 = sin proyecto).

03. Modelo entidad-relación (SQLite)



Modelo entidad-relación

Fuente editable: diagramas/fuentes/erd.mmd.

- **Base de datos:** archivo AsistenteIA.db3 en FileSystem.AppDataDirectory (carpeta privada de la app).
- **Biblioteca:** sqlite-net-pcl 1.9.172 con SQLitePCLRaw.bundle_green 2.1.10. Las tablas se crean con CreateTableAsync<T>(), que también agrega columnas nuevas a bases existentes.

Tablas

Tabla	Clave primaria	Claves foráneas	Descripción
Project	Id (INTEGER, autoincremental)	—	Materia, taller o trabajo que agrupa consultas.
ConversationRecord	Id (INTEGER, autoincremental)	ProjectId → Project.Id	Consulta guardada con su respuesta.

Tabla	Clave primaria	Claves foráneas	Descripción
		(**lógica**)	

Relación

Relación	Cardinalidad	Regla
Project agrupa ConversationRecord	1 : 0..*	ProjectId = 0 significa «sin proyecto».

Integridad referencial: sqlite-net no crea restricciones FOREIGN KEY. La integridad la mantiene DatabaseService.DeleteProjectAsync: antes de borrar un proyecto ejecuta UPDATE ConversationRecord SET ProjectId = 0 WHERE ProjectId = ?, así las consultas se conservan sin proyecto.

SQL equivalente (generado por sqlite-net)

```
CREATE TABLE "Project" (
  "Id" INTEGER PRIMARY KEY AUTOINCREMENT NOT NULL,
  "Name" VARCHAR, "Description" VARCHAR, "CreatedAt" BIGINT);

CREATE TABLE "ConversationRecord" (
  "Id" INTEGER PRIMARY KEY AUTOINCREMENT NOT NULL,
  "Prompt" VARCHAR, "Answer" VARCHAR, "CreatedAt" BIGINT,
  "Category" VARCHAR, "IsVerified" INTEGER, "ProjectId" INTEGER, "Note" VARCHAR);
```

04. Diccionario de datos

Las longitudes las valida la aplicación (ViewModels y PromptValidator); en SQLite las columnas VARCHAR no tienen límite. DateTime se guarda como *ticks* (BIGINT), que es el comportamiento por defecto de sqlite-net.

Tabla ConversationRecord (historial)

Campo	Tipo C#	Tipo SQLite	Restricciones	Valor por defecto	Función
Id	int	INTEGER	PK, autoincremental, no nulo	asignado por SQLite	Identificador. 0 = registro nuevo (insertar).
Prompt	string	VARCHAR	5–2000 caracteres (PromptValidator)	""	Consulta enviada a la IA.
Answer	string	VARCHAR	—	""	Respuesta recibida.
CreatedAt	DateTime	BIGINT	UTC	DateTime.UtcNow	Fecha; se muestra en hora local (dd/MM/yyyy HH:mm).
Category	string	VARCHAR	Categoría de la plantilla	"General"	Explicación, Resumen, Comparación, Estudio, Programación…
IsVerified	bool	INTEGER (0/1)	—	false	El estudiante contrastó la respuesta.
ProjectId	int	INTEGER	FK lógica a Project.Id; 0 = sin proyecto	0	Agrupa la consulta.
Note	string	VARCHAR	máx. 500 caracteres	""	Nota personal editable en el detalle.
ProjectName, CreatedAtLocal, VerificationText, MetaText	string	— ([Ignore])	No se guardan	—	Solo presentación.

Tabla Project

Campo	Tipo C#	Tipo SQLite	Restricciones	Valor por defecto	Función
Id	int	INTEGE	PK,	asignado por	Identificador.

Campo	Tipo C#	Tipo SQLite	Restricciones	Valor por defecto	Función
		R	autoincremental	SQLite	
Name	string	VARCHAR	3–60 caracteres, obligatorio	""	Nombre del proyecto.
Description	string	VARCHAR	máx. 300 caracteres	""	Contexto que se envía a la IA (máx. 400 caracteres incluido el nombre).
CreatedAt	DateTime	BIGINT	UTC	DateTime.UtcNow	Fecha de creación.
ConversationCount, CountText, DescriptionText	—	— ([Ignore])	No se guardan	—	Presentación.

Preferences (preferencias del dispositivo, sin cifrar)

Clave	Tipo	Ejemplo	Función
SelectedProvider	string	Gemini / Pollinations	Proveedor activo (cambio manual).
SelectedModel	string	gemini-3.5-flash-lite	Modelo de Gemini.
SelectedModel_Pollinations	string	openai	Modelo de Pollinations.
profile_name, profile_program, profile_institution	string (máx. 60)	—	Perfil del estudiante (no se envía a la IA).
responsible_use_notice_accepted	bool	true	El aviso de uso responsable ya se mostró.

SecureStorage (cifrado por el sistema operativo)

Clave	Contenido
ai_api_key	Clave de Google Gemini (nombre original, conserva claves guardadas antes).
pollinations_api_key	Clave de Pollinations.

05. Documentación de la API REST

La aplicación **consume** dos API REST de terceros (no expone una API propia). Ambas se llaman con un único HttpClient registrado como *singleton* (tiempo máximo **60 s**), método **POST**, cuerpo **JSON** y la clave **en un encabezado**, nunca en la URL.

5.1 Google Gemini — generateContent

Elemento	Valor
Método y URL	POST https://generativelanguage.googleapis.com/v1beta/models/{modelo}:generateContent
Parámetro de ruta	{modelo}: identificador del modelo (por defecto gemini-3.5-flash-lite). Si el usuario escribe models/x, se limpia a x.
Encabezados	x-goog-api-key: <clave> · Content-Type: application/json; charset=utf-8

Solicitud

```
{
  "systemInstruction": { "parts": [ { "text": "Eres un asistente académico. Responde de forma clara, indica la incertidumbre cuando exista y no inventes fuentes ni citas." } ] },
  "contents": [ { "role": "user", "parts": [ { "text": "Explica qué es una API REST" } ] } ],
  "generationConfig": { "maxOutputTokens": 4096 }
}
```

Campo	Obligatorio	Descripción
systemInstruction.parts[].text	No	Instrucción de sistema. Si la consulta tiene proyecto, se agrega «Contexto del proyecto…» (máx. 400 caracteres).
contents[].role / parts[].text	Sí	Turno del usuario con la consulta (5–2000 caracteres).
generationConfig.maxOutputTokens	No	Límite de longitud de la respuesta (4096).

Respuesta 200

```
{
  "candidates": [ {
    "content": { "parts": [ { "text": "Una API REST es …" } ] },
    "finishReason": "STOP"
  } ],
  "promptFeedback": { "blockReason": null }
}
```

Campo	Uso en la app
candidates[0].content.parts[].text	Se concatenan y se muestran.

Campo	Uso en la app
candidates[0].finishReason = "MAX_TOKENS"	Se agrega el aviso «Respuesta recortada por el límite de longitud».
promptFeedback.blockReason	Si no hay texto: «El servicio bloqueó la consulta (motivo)».

Respuesta de error

{ "error": { "code": 400, "message": "API key not valid. Please pass a valid API key.", "status": "INVALID_ARGUMENT" } }

5.2 Pollinations — chat/completions (compatible con OpenAI)

Elemento	Valor
Método y URL	POST https://gen.pollinations.ai/v1/chat/completions
Encabezados	Authorization: Bearer <clave> · Content-Type: application/json; charset=utf-8
Clave	Se crea en https://enter.pollinations.ai. La API pública de texto ya no permite uso anónimo sin límites.

Solicitud

```
{
 "model": "openai",
 "messages": [
  { "role": "system", "content": "Eres un asistente académico. ..." },
  { "role": "user", "content": "Explica qué es una API REST" }
 ],
 "max_tokens": 4096
}
```

Respuesta 200

{
 "choices": [{ "message": { "role": "assistant", "content": "Una API REST es ..." }, "finish_reason": "stop" }]
}

Campo	Uso en la app
choices[0].message.content	Texto mostrado.
choices[0].finish_reason = "length"	Aviso de respuesta recortada.
Error	Se lee {"error":{"message":...}}, {"error":"..."} o texto plano (máx. 200 caracteres).

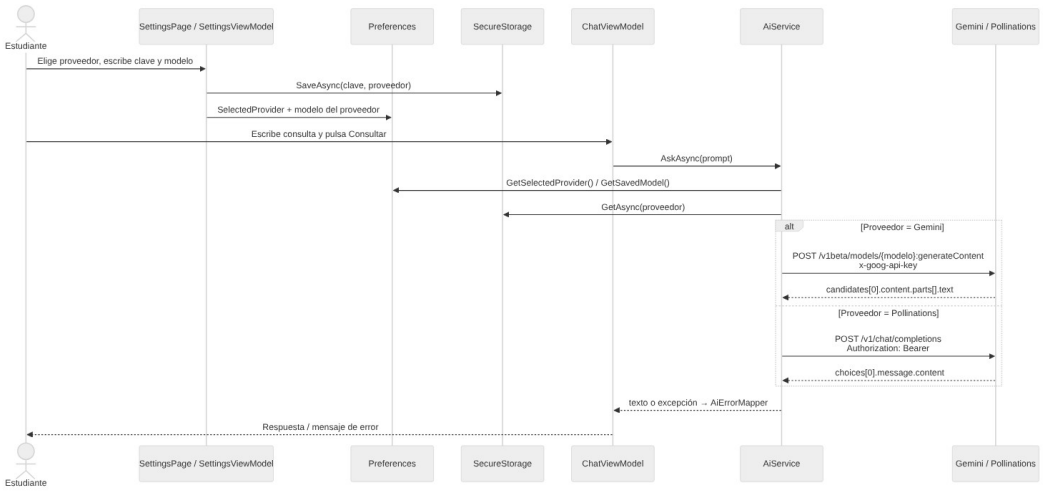
5.3 Códigos de estado

Código	Significado	Mensaje en la app
200	Correcto	Se muestra la respuesta y se habilita *Guardar*.
400	Solicitud inválida (en Gemini, también clave inválida)	«Error de autorización (400): la clave de API no es válida» o «Solicitud rechazada (400). Revise el modelo configurado».
401 / 403	Sin autorización / sin permisos	«Error de autorización (401/403): la clave de {proveedor} es inválida o no tiene permisos».
402	Sin saldo o límite gratuito (Pollinations)	«{proveedor} rechazó la solicitud por saldo o límite gratuito (402)…»
404	Modelo no encontrado	«Modelo no encontrado (404). Cambie el modelo en Configuración».
408	El servidor agotó su tiempo	«El servidor tardó demasiado en responder (408)…»
429	Cuota o límite de solicitudes	«Se superó la cuota o el límite de solicitudes (429)…»
5xx	Falla del proveedor	«El servicio de IA no está disponible (código)…»

El detalle completo del manejo de errores está en **08-Gestion-Errores**.

06. Arquitectura de proveedores de IA

Estado anterior: el PDF y el código solo usaban Google Gemini; Pollinations figuraba como alternativa descartada. **Estado actual:** la app soporta **Gemini y Pollinations** y el usuario cambia de proveedor **manualmente** en Configuración.



Flujo de selección y consulta

6.1 Diseño

Pieza	Archivo	Función
AiProvider (enum)	Models/ AiProvider.cs	Gemini = 0, Pollinations = 1.
AiProviderInfo	Models/ AiProvider.cs	Datos de cada proveedor: nombre, endpoint, modelo por defecto, modelos sugeridos, encabezado de clave, clave en SecureStorage y preferencia del modelo.
AiProviderCatalog	Models/ AiProvider.cs	Lista de proveedores, Parse (valor guardado → enum, con Gemini como respaldo), NormalizeModel, BuildEndpoint.
AiService	Services/ AiService.cs	AskAsync hace las validaciones comunes y llama a AskGeminiAsync o AskPollinationsAsync.
ChatModels	Models/ ChatModels.cs	DTO de Gemini (ChatRequest/ChatResponse) y de OpenAI/Pollinations (OpenAiChatRequest/OpenAiChatResponse).
SettingsViewModel / SettingsPage	ViewModels, Views	Selector de proveedor, modelos, clave, *Probar conexión*.

6.2 Comparación de proveedores

Aspecto	Google Gemini	Pollinations
Endpoint	/v1beta/models/ {modelo}:generateContent	/v1/chat/completions

Aspecto	Google Gemini	Pollinations
Formato	Propio de Google (contents, parts)	Compatible con OpenAI (messages)
Autenticación	x-goog-api-key	Authorization: Bearer
Modelo por defecto	gemini-3.5-flash-lite	openai
Modelos sugeridos	gemini-3.5-flash-lite, gemini-3.1-flash-lite, gemini-3.6-flash, gemini-3.8-flash	openai, openai-fast, mistral (lista vigente en gen.pollinations.ai/models)
Obtener clave	Google AI Studio	enter.pollinations.ai
Respuesta	candidates[0].content.parts[].text	choices[0].message.content

6.3 Selección y cambio manual

1. En **Perfil › Configuración**, el usuario elige el proveedor en el selector «Proveedor de IA».
2. Al cambiar el selector se cargan los modelos sugeridos y el estado de la clave **de ese proveedor**; aún no se usa en las consultas.
3. Al pulsar **Guardar** se guarda la clave (si se escribió una), el modelo en su preferencia (SelectedModel o SelectedModel_Pollinations) y el proveedor en SelectedProvider.
4. **Probar conexión** prueba el proveedor del selector aunque no se haya guardado.
5. Desde ese momento ChatViewModel muestra «Consultando {proveedor}…» y AiService usa el proveedor guardado. Cada proveedor conserva su propia clave y modelo, por lo que se puede alternar sin volver a escribirlos.

No hay cambio automático (*fallback*): si un proveedor falla, el mensaje de error sugiere cambiarlo manualmente. Así el usuario siempre sabe a qué servicio se envía su texto.

6.4 Cómo agregar otro proveedor

1. Agregar el valor al enum AiProvider y una entrada AiProviderInfo en AiProviderCatalog (endpoint, encabezado, claves).
2. Si usa formato OpenAI, reutilizar AskPollinationsAsync (o generalizarlo); si no, crear AskXxxAsync y sus DTO.
3. Agregar el caso en el switch de AiService.AskAsync.
4. Agregar pruebas en AiProviderCatalogTests. Las vistas no cambian.

07. Gestión de API Keys

7.1 Ciclo de vida

Paso	Dónde	Qué ocurre
Crear la clave	Google AI Studio / enter.pollinations.ai	El estudiante la crea con su cuenta.
Escribirla	Configuración → campo **Clave de API** (IsPassword=True, sin corrector ni predicción)	El texto se oculta mientras se escribe.
Guardarla	ApiKeyService.SaveAsync(clave, proveedor)	Se guarda en SecureStorage con Trim(). El campo se vacía.
Consultar estado	HasKeyAsync(proveedor)	La pantalla solo dice «Clave … guardada de forma segura» o «Sin clave configurada». **La clave nunca se vuelve a mostrar.**
Usarla	AiService.AskAsync	Se lee justo antes del POST y se envía en x-goog-api-key (Gemini) o Authorization: Bearer (Pollinations).
Probarla	Botón **Probar conexión**	Envía una consulta corta y muestra el resultado o el error (sin la clave).
Eliminarla	Botón **Eliminar clave** (con confirmación)	SecureStorage.Remove del proveedor seleccionado.

7.2 Dónde se guarda en cada plataforma

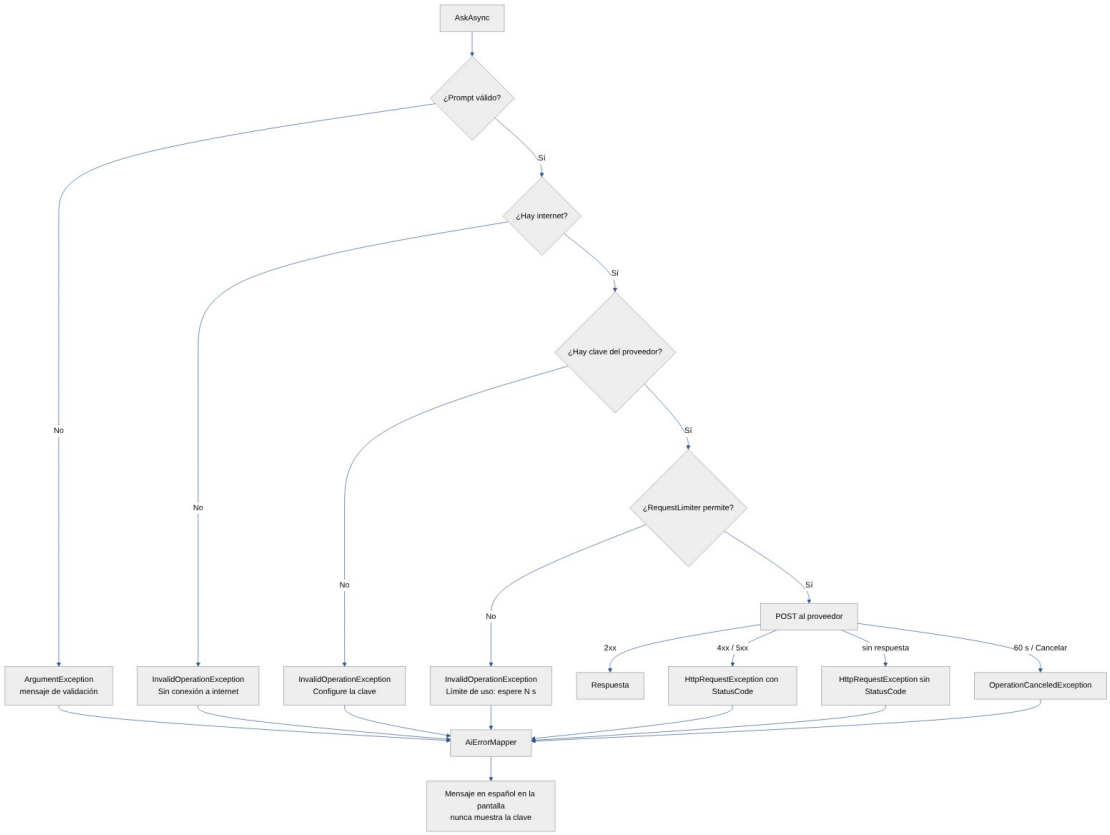
Plataforma	Mecanismo de SecureStorage (.NET MAUI)
Android	EncryptedSharedPreferences con clave en **Android Keystore**. Además android:allowBackup="false" evita copias de seguridad de los datos de la app.
Windows	Cifrado con **DataProtectionProvider** asociado al usuario de Windows (datos de la app no empaquetada).
iOS / Mac Catalyst	**Llavero (Keychain)**. *(Plataformas no compiladas en este proyecto.)*

7.3 Protección de credenciales

- Ninguna clave en el código, en appsettings, en el repositorio ni en capturas.
- La clave no viaja en la URL (no queda en historiales ni registros de proxy).
- Los mensajes de error se construyen con textos fijos (AiErrorMapper), nunca con la clave.
- SensitiveDataDetector avisa si el estudiante pega una clave (AIza…, sk-…) dentro de una consulta.
- Una clave independiente por proveedor (ai_api_key, pollinations_api_key).
- RequestLimiter (8 consultas por minuto) limita el gasto de cuota si alguien usa el equipo.

Análisis de riesgos y hallazgos: **10-Auditoria-Seguridad-API-Keys**.

08. Gestión de errores y excepciones



Flujo de errores

Todas las excepciones terminan en **AiErrorMapper.GetFriendlyMessage** (lógica pura, probada con 16 pruebas), que devuelve un mensaje en español **sin la clave**. La interfaz nunca se cierra: el ViewModel captura la excepción, muestra el mensaje en StatusMessage y deja los botones activos.

8.1 Errores antes de enviar

Situación	Excepción	Mensaje
Consulta vacía	ArgumentException	«Escriba una consulta antes de enviar.»
Menos de 5 / más de 2000 caracteres	ArgumentException	«La consulta es demasiado corta (mínimo 5 caracteres).» / «La consulta supera 2000 caracteres.»
Sin internet (Connectivity.NetworkAccess)	InvalidOperationException	«Sin conexión a internet. Verifique su red e intente de nuevo.»
Sin clave del proveedor	InvalidOperationException	«Configure la clave de {proveedor} en Perfil › Configuración.»
Más de 8 consultas por	InvalidOperationExc	«Límite de uso alcanzado: espere N s…»

Situación	Excepción	Mensaje
minuto	eption	
Datos sensibles detectados	(diálogo, no excepción)	El usuario decide *Enviar* o *Revisar*.

8.2 Errores HTTP

Código	Causa típica	Mensaje	Acción sugerida
400	Gemini: clave inválida («API key») o cuerpo/modelo inválido	«Error de autorización (400)…» o «Solicitud rechazada (400). Revise el modelo…»	Revisar clave o modelo.
401	Clave ausente o inválida	«Error de autorización (401): la clave de {proveedor} es inválida…»	Guardar una clave nueva.
402	Pollinations sin saldo o límite gratuito	«{proveedor} rechazó la solicitud por saldo o límite gratuito (402)…»	Revisar la cuenta o cambiar de proveedor.
403	Clave sin permisos / API no habilitada	«Error de autorización (403)…»	Revisar permisos del proyecto.
404	Modelo inexistente o retirado	«Modelo no encontrado (404). Cambie el modelo en Configuración.»	Elegir un modelo sugerido.
408	El servidor agotó su tiempo de espera	«El servidor tardó demasiado en responder (408)…»	Reintentar. *(Nuevo: no se manejaba.)*
429	Cuota o límite por minuto	«Se superó la cuota o el límite de solicitudes (429)…»	Esperar.
500–599	Falla del proveedor	«El servicio de IA no está disponible (código)…»	Reintentar más tarde o cambiar de proveedor.
Otro	—	«Error del servicio: código.»	—

8.3 Problemas de conexión y tiempo

Situación	Excepción	Mensaje
DNS, TLS o servidor inalcanzable	HttpRequestException sin StatusCode	«No se pudo conectar con el servicio. Verifique su conexión.»
Se superan los 60 s del HttpClient	TaskCanceledException / TimeoutException	«Tiempo de espera agotado. Intente de nuevo.»
El usuario pulsa **Cancelar**	OperationCanceledException con el token cancelado	«Consulta cancelada.»
Respuesta 200 sin texto	—	«El servicio no devolvió contenido.» o

Situación	Excepción	Mensaje
		«El servicio bloqueó la consulta (motivo) …»
Error al guardar en SQLite	Exception	«No se pudo guardar localmente: …»

09. Requisitos técnicos actualizados

Versiones tomadas de AsistenteIA.csproj, AsistenteIA.Tests.csproj y global.json.

9.1 SDK y plataformas

Elemento	Valor en el proyecto	Notas
.NET SDK	**9.0.100** (global.json, rollForward: latestFeature)	Acepta cualquier SDK 9.0.1xx o superior de la banda 9.0.
Frameworks de destino	net9.0-android · net9.0-windows10.0.19041.0	iOS y Mac Catalyst tienen carpeta en Platforms, pero no se compilan.
.NET MAUI	$(MauiVersion) (lo fija la carga de trabajo maui del SDK 9)	—
Windows (mínimo)	10.0.17763.0 (Windows 10 versión 1809)	WindowsPackageType = None (app no empaquetada).
Android (mínimo)	API 21 (Android 5.0)	Compilado con el SDK de Android del workload.
Pruebas	net9.0 (sin MAUI)	Se ejecutan en cualquier equipo con el SDK.

> ⚠️ **Fin de soporte:** Microsoft finaliza el soporte de **.NET 8 y .NET 9 el 10 de noviembre de 2026**. Se recomienda migrar a **.NET 10 (LTS, soporte hasta noviembre de 2028)**: cambiar los TFM a net10.0-… y global.json a 10.0.100. Ver **18-Conclusiones-Trabajo-Futuro**.

9.2 Herramientas

Herramienta	Versión mínima	Carga de trabajo / componente
Windows	10 (1809) u 11	—
Visual Studio 2022	17.12 o superior (compatible con .NET 9)	«Desarrollo de la interfaz de usuario de aplicaciones multiplataforma de .NET» (.NET MAUI)
Componentes adicionales	—	Herramientas de C++ para escritorio (requeridas por SQLitePCLRaw en Windows)
CLI (opcional)	dotnet workload install maui	Alternativa a Visual Studio
Android	Emulador API 34 o dispositivo con depuración USB	SDK de Android instalado por el workload
Git	2.30 o superior	—

9.3 Paquetes NuGet

Paquete	Versión	Proyecto	Uso
Microsoft.Maui.Controls	$(MauiVersion)	App	Marco de interfaz.
Microsoft.Extensions.Logging.Debug	9.0.0	App	Registro en depuración.
CommunityToolkit.Mvvm	8.3.2	App	[ObservableProperty], [RelayCommand].
sqlite-net-pcl	1.9.172	App	ORM SQLite.
SQLitePCLRaw.bundle_green	2.1.10	App	Motor SQLite nativo.
Microsoft.NET.Test.Sdk	17.11.1	Pruebas	Ejecución de pruebas.
xunit	2.9.2	Pruebas	Marco de pruebas.
xunit.runner.visualstudio	2.8.2	Pruebas	Explorador de pruebas de VS.

9.4 Servicios externos

Servicio	Requisito
Google Gemini	Clave de Google AI Studio y un modelo vigente (por defecto gemini-3.5-flash-lite).
Pollinations	Clave de enter.pollinations.ai y un modelo de texto vigente (por defecto openai).
Red	Internet con salida HTTPS (443) a generativelanguage.googleapis.com y gen.pollinations.ai.

10. Auditoría de seguridad de API Keys

Alcance: almacenamiento, transmisión, visualización y uso de las claves de Gemini y Pollinations en el código del 3 de noviembre de 2026. **Método:** revisión del código fuente, del repositorio y de la configuración del proyecto.

10.1 Verificación de controles

#	Control	Evidencia en el código	Resultado
C1	No hay claves en el código	Búsqueda de AIza..., sk-..., pk_..., sk_... en todo el historial: solo aparecen dos claves **ficticias** dentro de las pruebas	✅ Cumple
C2	No hay claves en el repositorio ni en el historial Git	Revisión de los 6 commits; sin archivos de configuración con secretos	✅ Cumple
C3	Almacenamiento cifrado	ApiKeyService → SecureStorage.Default	✅ Cumple
C4	Campo oculto	Entry IsPassword="True", sin corrector ni predicción	✅ Cumple
C5	La clave no se vuelve a mostrar	El ViewModel vacía ApiKey tras guardar; solo se informa si existe	✅ Cumple
C6	La clave no viaja en la URL	x-goog-api-key / Authorization: Bearer	✅ Cumple
C7	Solo HTTPS	Endpoints https://; Android usesCleartextTraffic="false"	✅ Cumple
C8	Errores sin la clave	AiErrorMapper usa textos fijos; prueba Messages_NeverContainTheKey	✅ Cumple
C9	Sin copias de seguridad en Android	android:allowBackup="false"	✅ Cumple
C10	Eliminación por el usuario	Botón *Eliminar clave* con confirmación	✅ Cumple
C11	Límite de uso	RequestLimiter: 8 solicitudes/min	✅ Cumple
C12	Aviso al pegar una clave en una consulta	SensitiveDataDetector detecta AIza... y sk-...	✅ Cumple
C13	Clave separada por proveedor	ai_api_key / pollinations_api_key	✅ Cumple
C14	Registros (logs) sin la clave	No se registran encabezados; AddDebug solo en DEBUG	✅ Cumple

10.2 Matriz de riesgos de credenciales

Riesgo	Prob.	Impacto	Nivel	Mitigación implementada	Riesgo residual
Clave publicada en GitHub	Baja	Alto	Medio	C1, C2, .gitignore	Bajo
Clave visible en capturas o video	Media	Alto	Alto	C4, C5	Bajo
Robo del archivo de la app / respaldo	Baja	Alto	Medio	C3, C9	Bajo
Interceptación en red	Baja	Alto	Medio	C6, C7	Bajo
Clave en registros o mensajes	Baja	Medio	Bajo	C8, C14	Bajo
Gasto de cuota por uso indebido del equipo	Media	Medio	Medio	C11	Medio (el equipo desbloqueado sigue pudiendo consultar)
Clave pegada en una consulta y enviada al proveedor	Media	Alto	Alto	C12 (aviso con confirmación)	Medio (es una heurística)
Ingeniería inversa del APK para extraer claves	—	—	—	No aplica: la app no incluye claves; cada usuario usa la suya	Nulo

10.3 Hallazgos y recomendaciones

ID	Hallazgo	Severidad	Recomendación
H1	La clave se lee en memoria como string y permanece hasta que el recolector la libera.	Baja	Aceptable en una app cliente; no guardarla en campos estáticos (ya se cumple).
H2	En Windows no empaquetado, el cifrado depende de la cuenta de usuario de Windows.	Baja	Recomendar cuentas con contraseña y no compartir el usuario.
H3	No hay rotación ni caducidad automática de claves.	Baja	Indicar en el manual cómo revocar la clave en Google AI Studio / Pollinations si se filtra.
H4	Las claves son personales y se usan desde el cliente (sin servidor	Media (si se publica	Para uso masivo, usar un backend que guarde la clave y aplique cuotas

ID	Hallazgo	Severidad	Recomendación
	intermedio).	la app)	por usuario.
H 5	Las dependencias pueden tener vulnerabilidades nuevas.	Baja	Ejecutar dotnet list package --vulnerable antes de cada entrega.

Conclusión: no se encontraron claves expuestas y los 14 controles se cumplen. Los riesgos residuales son bajos para un proyecto académico de uso personal.

11. Manual de usuario (actualizado)

AsistenteIA es un asistente académico: envía tus preguntas a un modelo de inteligencia artificial, muestra la respuesta y guarda un historial **solo en tu dispositivo**. Funciona en Windows y Android.

11.1 Primer uso

1. Abre la app. La primera vez aparece el aviso **«Uso responsable de la IA»**; léelo y pulsa **Entendido**.
2. En **Inicio** pulsa **Configurar clave y modelo** (o ve a **Perfil › Configuración**).
3. Elige el **proveedor**: *Google Gemini* o *Pollinations*.
4. Escribe tu **clave de API** (se oculta) y elige un **modelo** sugerido.
5. Pulsa **Guardar** y luego **Probar conexión**. Debe aparecer «✔️ Conexión exitosa con {proveedor}».

Clave de Gemini: Google AI Studio (aistudio.google.com/apikey). Clave de Pollinations: enter.pollinations.ai.

11.2 Pantallas

Pantalla	Para qué sirve	Acciones
Inicio	Bienvenida con tu nombre, accesos rápidos y recomendaciones.	Nueva consulta, Mis proyectos, Historial, Mi perfil, Configurar clave y modelo.
Consultar	Hacer preguntas a la IA.	Elegir plantilla y proyecto, escribir (contador hasta 2000), **Consultar**, **Cancelar**, **Guardar**.
Proyectos	Agrupar consultas por materia o trabajo.	**+ Nuevo proyecto**, abrir un proyecto.
Detalle de proyecto	Ver y editar un proyecto.	Editar nombre (3–60) y descripción (máx. 300), ver sus consultas, **Consultar en este proyecto**, eliminar.
Historial	Ver consultas guardadas.	Buscar (en pregunta, respuesta, categoría o proyecto), **Solo verificadas**, abrir detalle, **Eliminar**.
Detalle de consulta	Revisar una consulta.	**Marcar como verificada / Quitar verificación**, escribir una **nota personal** (máx. 500) y pulsar **Guardar nota**, **Eliminar consulta**.
Perfil	Tus datos y estadísticas.	Nombre, programa e institución; contadores de consultas, verificadas y proyectos; Configuración; **Borrar historial de consultas**; **Borrar todos mis datos locales**.
Configuración	Proveedor, clave y modelo.	Selector de proveedor, clave oculta, modelos, Guardar, Probar conexión, Eliminar clave; avisos

Pantalla	Para qué sirve	Acciones
		de privacidad y «Acerca de».

11.3 Hacer una consulta

1. Ve a **Consultar**.
2. (Opcional) Elige una **plantilla**: Pregunta libre, Explicar un concepto, Resumir un texto, Comparar dos temas, Preguntas de práctica o Revisar código. El texto aparece en el editor y puedes cambiarlo.
3. (Opcional) Elige un **proyecto**: su nombre y descripción se envían como contexto.
4. Escribe tu pregunta (5 a 2000 caracteres) y pulsa **Consultar**. Verás «Consultando {proveedor}…».
5. Si tarda, pulsa **Cancelar**. La espera máxima es 60 segundos.
6. Lee la respuesta, **verifícala** con fuentes confiables y pulsa **Guardar** para añadirla al historial.

Avisos que puedes ver

Aviso	Qué significa
«Posibles datos sensibles…»	Tu texto parece tener un correo, teléfono, cédula, tarjeta, contraseña o clave. Pulsa **Revisar** para quitarlos o **Enviar** si estás seguro.
«Límite de uso alcanzado: espere N s»	Hiciste más de 8 consultas en un minuto.
«[Respuesta recortada…]»	La respuesta superó el límite de longitud: pide un resumen o divide la pregunta.

11.4 Cambiar de proveedor

En **Configuración** elige el otro proveedor, escribe su clave (solo la primera vez), elige el modelo y pulsa **Guardar**. Cada proveedor guarda su propia clave y su propio modelo. La línea «Proveedor activo» muestra cuál se está usando.

11.5 Privacidad y datos

- Tu clave se guarda cifrada en el dispositivo y nunca se muestra.
- El historial, los proyectos y el perfil se guardan solo en tu dispositivo. El perfil **no** se envía a la IA.
- Lo único que se envía al proveedor es el texto de la consulta (y el contexto del proyecto, si elegiste uno).
- **Borrar historial de consultas** elimina las consultas; **Borrar todos mis datos locales** elimina consultas, proyectos y perfil. La clave se elimina en Configuración.

11.6 Recomendaciones

- No escribas datos personales, contraseñas ni información confidencial.

- La IA puede equivocarse o inventar datos: márcala como verificada solo después de contrastarla.
- Si ves un error, consulta el **Manual de solución de problemas**.

12. Manual técnico para desarrolladores

12.1 Organización del código

```
AsistenteIA.slnx
├── AsistenteIA/                # App .NET MAUI (Windows, Android)
│   ├── MauiProgram.cs          # Inyección de dependencias y HttpClient (60 s)
│   ├── AppShell.xaml(.cs)      # TabBar de 5 pestañas + rutas Detail, ProjectDetail, Settings
│   ├── Models/                 # ChatModels (DTO Gemini y OpenAI), ConversationRecord, Project,
│   │                           # PromptTemplate, UserProfile, AiProvider (catálogo de proveedores)
│   ├── Services/               # AiService, AiErrorMapper, ApiKeyService, DatabaseService, ProfileService,
│   │                           # PromptValidator, SensitiveDataDetector, RequestLimiter,
│   │                           # INavigationService, IDialogService
│   ├── ViewModels/             # 8 ViewModels con CommunityToolkit.Mvvm
│   ├── Views/                  # 8 páginas XAML (solo binding)
│   ├── Resources/              # Colors.xaml, Styles.xaml, fuentes, íconos
│   └── Platforms/              # Android (manifest), Windows, iOS, MacCatalyst
├── AsistenteIA.Tests/          # xUnit (net9.0) — enlaza los archivos de lógica pura
└── docs/                       # Documentación (01–07 y complementaria/)
```

12.2 Patrones y reglas

Regla	Detalle
MVVM	Las vistas no tienen lógica; los ViewModels no usan `Shell` directamente (usan `INavigationService` e `IDialogService`).
Inyección	Servicios *singleton*; ViewModels y páginas *transient*; las páginas reciben su ViewModel por constructor.
Asincronía	Toda E/S es `async`; las consultas aceptan `CancellationToken`.
Lógica probable	Lo que no depende de MAUI va en clases puras y se enlaza en `AsistenteIA.Tests.csproj` con `<Compile Include … Link=…>`.
Configuración	Proveedor y modelo en `Preferences`; claves en `SecureStorage`; nada en el código.

12.3 Configuración y ejecución

```
git clone https://github.com/sebitazz321/ASISTENTEIA.git
cd ASISTENTEIA
dotnet workload install maui          # si no usa Visual Studio
dotnet build AsistenteIA/AsistenteIA.csproj -f net9.0-windows10.0.19041.0
dotnet build AsistenteIA/AsistenteIA.csproj -f net9.0-android
```

En Visual Studio: abrir `AsistenteIA.slnx`, elegir **Windows Machine** (o un emulador Android) y pulsar **Ejecutar**. El ejecutable de depuración queda en `AsistenteIA/bin/Debug/net9.0-windows10.0.19041.0/win10-x64/AsistenteIA.exe` (la arquitectura puede variar).

12.4 Pruebas

```
dotnet test AsistenteIA.Tests       # 66 pruebas: validador, datos sensibles, límite, perfil, proveedores, errores
```

Clase	Casos	Cubre
PromptValidatorTests	8	RF02
SensitiveDataDetectorTests	18	Privacidad
RequestLimiterTests	3	Límite de uso
UserProfileTests	5	Iniciales del avatar
AiProviderCatalogTests	16	Catálogo, parseo, modelos, endpoints, claves en encabezado
AiErrorMapperTests	16	400, 401, 402, 403, 404, 408, 429, 5xx, conexión, cancelación, sin clave en mensajes

Las pruebas manuales P01–P15 están en docs/03-Plan-de-pruebas.md.

12.5 Mantenimiento

Tarea	Cómo
Cambiar los modelos sugeridos	AiProviderCatalog (SuggestedModels, DefaultModel) y su prueba DefaultModel_IsAmongSuggested.
Agregar un proveedor	Ver **06-Arquitectura-Proveedores-IA §6.4**.
Agregar un campo a SQLite	Propiedad en la entidad; CreateTableAsync agrega la columna a bases existentes (no borra datos).
Cambiar el límite de uso	RequestLimiter.DefaultMaxRequests / DefaultWindow.
Cambiar el tiempo máximo	Timeout del HttpClient en MauiProgram.cs.
Actualizar paquetes	dotnet list package --outdated y --vulnerable; probar y registrar en el historial de cambios.
Migrar a .NET 10	TFM net10.0-android;net10.0-windows10.0.19041.0, global.json 10.0.100, actualizar paquetes y ejecutar las pruebas.
Entregar sin binarios	git archive --format=zip --output=../AsistenteIA-entrega.zip HEAD

13. Manual de solución de problemas

13.1 Instalación

Problema	Causa probable	Solución
Visual Studio no muestra proyectos MAUI	Falta la carga de trabajo	Visual Studio Installer → Modificar → «Desarrollo de la interfaz de usuario de aplicaciones multiplataforma de .NET».
NETSDK1045 / «el SDK actual no admite .NET 9»	SDK antiguo	Instalar .NET SDK 9.0.1xx o superior; dotnet --list-sdks para verificar.
A compatible .NET SDK was not found (global.json)	No hay SDK 9.0	Instalar el SDK 9 o, si migró a .NET 10, actualizar global.json.
workload maui not installed	Falta el workload por consola	dotnet workload install maui (como administrador).
No aparecen emuladores Android	SDK/emulador no instalado	Herramientas → Android → Administrador de dispositivos → crear un dispositivo API 34.

13.2 Compilación

Problema	Causa probable	Solución
Unable to load DLL 'e_sqlite3' en Windows	Faltan componentes C++	Instalar «Herramientas de compilación de C++ para escritorio» y recompilar.
Errores de paquetes NuGet tras cambiar de equipo	Caché o restauración incompleta	Clic derecho en la solución → Restaurar paquetes NuGet; o dotnet restore.
Errores raros después de cambiar de rama	Carpetas bin/obj viejas	Cerrar VS, borrar bin y obj de ambos proyectos, compilar de nuevo.
XFC0000 / errores XAML	Binding o recurso mal escrito	Revisar la línea indicada; los estilos están en Resources/Styles.
Las pruebas no aparecen en el Explorador	Falta compilar el proyecto de pruebas	Compilar AsistenteIA.Tests y pulsar «Ejecutar todas»; o dotnet test AsistenteIA.Tests.
Archivo de la app en uso al compilar	La app sigue abierta	Cerrar la app o detener la depuración.

13.3 Conexión con la IA

Mensaje	Causa	Solución
«Sin conexión a internet…»	Sin red	Revisar Wi-Fi/datos y volver a intentar.
«No se pudo conectar con	DNS, proxy o firewall	Permitir

Mensaje	Causa	Solución
el servicio…»	bloquean el dominio	generativelanguage.googleapis.com y gen.pollinations.ai (puerto 443).
«Tiempo de espera agotado…»	Respuesta muy lenta (> 60 s)	Reintentar, acortar la consulta o cambiar de modelo/proveedor.
«Error de autorización (400/401/403)…»	Clave inválida, revocada o de otro proveedor	Configuración → elegir el proveedor correcto → pegar una clave nueva → Guardar → Probar conexión.
«…saldo o límite gratuito (402)»	Pollinations sin saldo	Revisar la cuenta en enter.pollinations.ai o cambiar a Gemini.
«Modelo no encontrado (404)…»	Modelo retirado o mal escrito	Elegir un modelo de la lista sugerida.
«Se superó la cuota… (429)»	Límite del proveedor	Esperar unos minutos.
«Límite de uso alcanzado: espere N s»	Más de 8 consultas por minuto (límite de la app)	Esperar el tiempo indicado.
«El servicio de IA no está disponible (5xx)»	Falla del proveedor	Reintentar más tarde o cambiar de proveedor.
«El servicio bloqueó la consulta (…)»	Filtro de seguridad de Gemini	Reformular la pregunta.

13.4 Configuración y datos

Problema	Causa	Solución
«Configure la clave de {proveedor}…»	El proveedor activo no tiene clave	Configuración → escribir la clave de ese proveedor → Guardar.
Cambié de proveedor y sigue usando el anterior	No se pulsó Guardar	El selector solo prepara el cambio; pulse **Guardar** y revise «Proveedor activo».
«No se pudo leer el almacén seguro»	SecureStorage dañado (p. ej. tras restaurar el sistema)	Eliminar la clave y guardarla de nuevo; en Android, borrar datos de la app.
El historial desapareció	Se usó «Borrar todos mis datos locales» o se reinstaló la app	Los datos son locales; no hay copia en la nube.
El aviso de uso responsable no vuelve a salir	Ya se aceptó	Es normal; se muestra una sola vez.

14. Actualización de tecnologías utilizadas

Corrige y completa la tabla «1. Tecnologías» del documento (que no tenía versiones). Versiones verificadas en el código y en las fuentes oficiales el 3 de noviembre de 2026.

Componente	Documento original	Versión real / corrección	Uso
.NET SDK	«.NET 9 o superior»	**9.0.100** (global.json, latestFeature) · ⚠️ fin de soporte 10/11/2026 → migrar a .NET 10 LTS	Compilación.
.NET MAUI	Visual Studio y .NET MAUI	Workload maui de .NET 9 ($(MauiVersion))	Interfaz multiplataforma.
Plataformas	Windows o Android	net9.0-windows10.0.19041.0 (mín. 10.0.17763) y net9.0-android (mín. API 21)	Destinos.
C# y XAML	C# y XAML	C# 13 (por defecto en .NET 9): *collection expressions* y expresiones switch con patrones	Lógica e interfaz.
MVVM	CommunityToolkit.Mvvm	**8.3.2**	[ObservableProperty], [RelayCommand].
Shell	Shell	TabBar de 5 pestañas + 3 rutas secundarias	Navegación.
HTTP/JSON	HttpClient y System.Text.Json	HttpClient singleton (60 s) · System.Net.Http.Json · JsonDocument para errores	Consumo REST.
SQLite	sqlite-net-pcl	**1.9.172** + SQLitePCLRaw.bundle_green **2.1.10**	Historial y proyectos.
Almacenamiento seguro	SecureStorage	SecureStorage de MAUI (Keystore / DataProtection) + Preferences	Claves y preferencias.
Registro	—	Microsoft.Extensions.Logging.Debug **9.0.0** (solo DEBUG)	Depuración.
Pruebas	Proyecto AsistenteIA.Tests	**xUnit 2.9.2**, xunit.runner.visualstudio 2.8.2, Microsoft.NET.Test.Sdk 17.11.1	66 pruebas.
IA – Google Gemini	Gemini, por defecto gemini-3.5-flash-lite	API v1beta generateContent. Modelos estables: gemini-3.5-flash-lite, gemini-3.1-flash-lite, gemini-3.6-flash, gemini-3.8-flash (todos vigentes; gemini-2.0-* y gemini-3.1-flash-lite-preview ya fueron retirados)	Proveedor 1.
IA – Pollinations	Alternativa no usada	**Agregado:** gen.pollinations.ai/v1/chat/completions	Proveedor 2.

Componente	Documento original	Versión real / corrección	Uso
		(formato OpenAI), clave en Authorization: Bearer, modelo por defecto openai	
Control de versiones	Git (rama main)	Git + GitHub sebitazz321/ASISTENTEIA	Repositorio.
Asistencia de IA para desarrollo	Claude (Anthropic)	Claude (Anthropic) — declarado en §8.2 del documento	Apoyo en código y documentación.

Fuentes

- Modelos Gemini: https://ai.google.dev/gemini-api/docs/models
- Pollinations API: https://gen.pollinations.ai/docs y https://github.com/pollinations/pollinations/blob/master/APIDOCS.md
- Fin de soporte de .NET 8 y 9: https://devblogs.microsoft.com/dotnet/dotnet-8-9-end-of-support/
- Política de soporte de .NET MAUI: https://dotnet.microsoft.com/platform/support/policy/maui

15. Actualización de funcionalidades

Comparación entre lo que describe el documento (§2.2 requisitos funcionales y §4 interfaz) y lo que realmente implementa el código.

15.1 Requisitos del documento

Código	Requisito	Documento	Código real	Observación
RF01	Ingresar pregunta	Cumple	✅	Editor con contador «N / 2000».
RF02	Validar contenido y longitud	Cumple	✅	5–2000 caracteres (PromptValidator).
RF03	POST a la API REST	Cumple	✅ ampliado	Ahora a Gemini **o** Pollinations.
RF04	Estados de carga, respuesta, error, sin conexión	Cumple	✅ ampliado	Mensajes 402 y 408 agregados.
RF05	Guardar en SQLite	Cumple	✅	Botón Guardar.
RF06	Consultar, filtrar, detalle, eliminar	Cumple	✅	Búsqueda también en respuesta, categoría y proyecto.
RF07	Plantillas editables	Cumple	✅	6 plantillas.
RF08	Clave sin mostrarla	Cumple	✅ ampliado	Una clave por proveedor.
RF09	Acerca de	Cumple	✅ corregido	La «Autoría» tenía un texto de plantilla; ahora tiene los nombres.
RF10	Windows o Android	Cumple	✅	Probado en Windows según el documento.

15.2 Funcionalidades implementadas que el documento no describía

ID	Funcionalidad	Dónde	Estado
RF11	**Proveedores Gemini y Pollinations con cambio manual**	Configuración, AiService	**Nuevo** (3/11/2026)
RF12	Aviso de uso responsable al primer inicio	Inicio (HomeViewModel)	Ya existía en el zip
RF13	Aviso de datos sensibles antes de enviar (correo, tarjeta, teléfono/cédula, credenciales, claves)	Consultar (SensitiveDataDetector)	Ya existía
RF14	Límite de 8 consultas por minuto	RequestLimiter	Ya existía
RF15	Contexto del proyecto enviado a la IA	Consultar	Ya existía
RF16	Aviso de respuesta recortada (límite de 4096	AiService	Ya existía

ID	Funcionalidad	Dónde	Estado
	tokens)		
RF17	Nota personal por consulta	Detalle de consulta	Ya existía
RF18	Detalle de proyecto: editar, ver sus consultas, consultar en el proyecto, eliminar conservando consultas	Detalle de proyecto	Ya existía
RF19	Estadísticas del perfil (consultas, verificadas, proyectos)	Perfil	Ya existía (el documento lo menciona en §4.5)
RF20	Borrar historial / borrar todos los datos locales	Perfil	Ya existía
RF21	Mensajes de error 402 y 408	AiErrorMapper	**Nuevo**

15.3 Diferencias en pruebas y pantallas

Tema	Documento	Real
Casos de prueba manuales	«Se ejecutaron los diez casos» pero la tabla tiene **nueve** (P01–P09); la lista de cotejo dice «Nueve casos con evidencia».	docs/03-Plan-de-pruebas.md define **P01–P15**. Falta ejecutar y documentar P10–P15.
Pruebas automáticas	34	**66** (34 + 32 nuevas de proveedores y errores). Pendiente: ejecutarlas en Visual Studio y actualizar la Figura 18.
Pantallas	7 en la lista de cotejo	**8** (falta «Detalle de proyecto»).
Configuración	Clave y modelo	Proveedor + clave + modelo por proveedor.

15.4 Casos de prueba nuevos sugeridos

Caso	Condición	Resultado esperado
P16	Cambiar a Pollinations, guardar clave y probar conexión	«✔️ Conexión exitosa con Pollinations».
P17	Consultar con Pollinations	La respuesta aparece y el estado dice «Respuesta recibida de Pollinations».
P18	Volver a Gemini sin escribir la clave otra vez	Funciona: cada proveedor conserva su clave.
P19	Cambiar el selector sin pulsar Guardar y consultar	Se sigue usando el proveedor activo anterior.
P20	Clave inválida en Pollinations	«Error de autorización (401)…» sin mostrar la clave.

16. Historial de cambios del proyecto

Construido a partir del historial Git (git log) y del contenido de cada versión.

Versión	Fecha	Commit	Autor registrado en Git	Cambios	Archivos afectados
0.1	22/10/2026	4fc6329	Juan Rodriguez	Agrega .gitattributes y .gitignore.	2 archivos
0.2	22/10/2026	5844a08	Juan Rodriguez	Agrega los archivos del proyecto (plantilla .NET MAUI).	38 archivos
0.3	22/10/2026	20b0bb9	Juan Rodriguez	«Trabajo completo»: modelos, servicios, ViewModels y vistas iniciales.	56 archivos
1.0	03/11/2026	17ab990	Sebastian Grande Moreno	Integra la versión mejorada entregada en el zip (no estaba versionada): Proyectos, Detalle de proyecto, Perfil, navegación y diálogos inyectables, datos sensibles, límite de uso, notas, 34 pruebas xUnit, docs 01–07. Cierra el *merge* pendiente entre 20b0bb9 y 5844a08.	63 archivos (34 nuevos, 2 eliminados, 27 modificados)
1.1	03/11/2026	fb45b32	Sebastian Grande Moreno	Proveedor Pollinations y cambio manual; clave por proveedor; AiErrorMapper con 402 y 408; selector en Configuración; nombres en «Acerca de».	AiProvider.cs, ChatModels.cs, AiErrorMapper.cs, AiService.cs, ApiKeyService.cs, ChatViewModel.cs, SettingsViewModel.cs, SettingsPage.xaml
1.2	03/11/2026	76bd256	Sebastian Grande Moreno	32 pruebas nuevas (total 66).	AiProviderCatalogTests.cs, AiErrorMapperTests.cs, AsistenteIA.Tests.csproj
1.3	03/11/20	(este comm	Sebastian Grande	Documentación complementaria (18	docs/ complementaria/**

Versión	Fecha	Commit	Autor registrado en Git	Cambios	Archivos afectados
	26	it)	Moreno	entregables), diagramas y comparación con el documento.	

Responsables por área (según §11 del documento)

Integrante	Áreas
Juan Carlos Díaz Cifuentes	API REST, almacenamiento seguro de la clave, SQLite, pruebas y documentación.
Sebastián Grande Moreno	Navegación, pantallas XAML, estilos, integración de la solución y repositorio.

> Nota: el autor que Git registra en los tres primeros commits es «Juan Rodriguez <juandavidrj037@…>» y el *remote* original era JDRODRIGUEZ037/AsistenteIA. Si corresponde a la cuenta de un integrante, conviene aclararlo en la sustentación; si no, el equipo debe explicar el origen de ese código.

17. Evidencia del repositorio Git

17.1 Estado encontrado (antes de esta actualización)

Elemento	Hallazgo
Repositorio en GitHub sebitazz321/ASISTENTEIA	**Vacío** (sin commits).
Carpeta .git dentro del zip	3 commits del 22/10/2026, autor «Juan Rodriguez», *remote* https://github.com/JDRODRIGUEZ037/AsistenteIA.git.
Ramas	main local (20b0bb9) y origin/main (5844a08) habían divergido y había un ***merge* sin terminar** (MERGE_HEAD).
Cambios sin versionar	60 archivos modificados, nuevos o eliminados (todo el trabajo de mejora).
Coincidencia con el documento (§11)	**No coincide**: el documento afirma «commits descriptivos por funcionalidad realizados por los dos integrantes».

17.2 Estado actual

- **Remoto:** https://github.com/sebitazz321/ASISTENTEIA · **rama:** main.
- Se conservó el historial original (no se reescribió) y se agregaron commits descriptivos:

```
*   (docs)   Documentación complementaria: 18 entregables y comparación   ← este commit
*   76bd256   Agrega 32 pruebas de proveedores y manejo de errores
*   fb45b32   Agrega proveedor Pollinations y cambio manual de proveedor de IA
*   17ab990   Integra la versión mejorada del proyecto (AsistenteIA_mejorado)   (merge de 20b0bb9 y 5844a08)
|\
| * 5844a08   Agregar archivos de proyecto.
* | 20b0bb9   Trabajo completo Agregar archivos de proyecto.
|/
*   4fc6329   Agregar .gitattributes y .gitignore.
```

17.3 Organización del repositorio

Ruta	Contenido
AsistenteIA/	Aplicación .NET MAUI
AsistenteIA.Tests/	Pruebas xUnit
docs/01…07	Documentación original del equipo (Markdown)
docs/complementaria/	Los 18 entregables, diagramas (PNG + Mermaid), comparación, matriz Excel y documento Word/PDF consolidado
.gitignore	Excluye bin/, obj/, .vs/, *.user

17.4 Flujo recomendado para el resto del proyecto

1. Cada integrante configura su identidad: git config user.name "Nombre Apellido" y git config user.email "correo".

2. Trabajo en ramas cortas: feature/..., fix/..., docs/..., con *Pull Request* hacia main.
3. Un commit por cambio lógico, en imperativo y explicando el porqué.
4. Comandos útiles para la evidencia:

```
git log --graph --oneline --all        # gráfico de ramas y commits
git shortlog -sne                      # commits por autor
git log --stat --since="2026-10-01"    # archivos afectados por commit
```

18. Conclusiones y trabajo futuro

18.1 Resultados obtenidos

- Aplicación .NET MAUI funcional en Windows (y compilable para Android) con **8 pantallas**, arquitectura **MVVM** e inyección de dependencias.
- Consumo de API REST de IA con **POST + JSON**, cancelación, tiempo máximo de 60 s y manejo de errores por código HTTP.
- **Dos proveedores** (Gemini y Pollinations) con cambio manual, sin tocar vistas ni ViewModels de consulta.
- Persistencia **SQLite** con CRUD de consultas y proyectos; perfil y preferencias locales.
- Seguridad de credenciales: SecureStorage, clave en encabezado, sin claves en el repositorio (auditoría con 14 controles cumplidos).
- Uso responsable: aviso inicial, detector de datos sensibles, límite de uso, verificación manual y borrado de datos.
- **66 pruebas automáticas** sobre la lógica pura y 15 casos manuales definidos.

18.2 Cumplimiento de objetivos

Objetivo	Resultado
Asistente académico con IA vía API REST	✅
Historial persistente en SQLite	✅
Seguridad de la clave	✅
Mínimo cinco pantallas con Shell y MVVM	✅ (8)
Manejo de carga, cancelación y errores	✅
Pruebas	✅ automáticas · ⚠️ manuales P10–P15 y P16–P20 por ejecutar
Evidencia de trabajo en equipo en Git	⚠️ El historial no refleja aún commits de ambos integrantes

18.3 Limitaciones

1. El código nuevo (Pollinations, errores 402/408 y 32 pruebas) se escribió sin compilador disponible en el entorno de edición: **debe compilarse y ejecutarse en Visual Studio** antes de la entrega.
2. .NET 9 deja de tener soporte el 10/11/2026.
3. Los modelos y la API de Pollinations cambian con frecuencia; los modelos sugeridos deben revisarse.
4. Sin sincronización entre dispositivos: el historial es solo local.
5. `SensitiveDataDetector` es heurístico (puede tener falsos positivos y negativos).
6. Las pantallas no tienen pruebas automáticas de interfaz.
7. `ApplicationId` sigue con el valor de plantilla `com.companyname.asistenteia`.

18.4 Trabajo futuro

Prioridad	Mejora
Alta	Migrar a **.NET 10 LTS** y actualizar paquetes (CommunityToolkit.Mvvm, xUnit).
Alta	Ejecutar y documentar con capturas los casos P10–P20 y las 66 pruebas.
Media	Respuestas en *streaming* para ver el texto mientras se genera.
Media	Conversaciones de varios turnos (enviar el historial de la conversación como contexto).
Media	Cambio automático de proveedor opcional ante 5xx/429, siempre informando al usuario.
Media	Exportar el historial a PDF o Markdown.
Media	Pruebas de interfaz y pruebas de AiService con un HttpMessageHandler simulado.
Baja	Publicar en Microsoft Store / Google Play con un ApplicationId propio.
Baja	Backend propio para no exponer claves si la app se distribuye a muchos usuarios.